



Evolutionary Computation

Assignment 6: MSLS and ILS

BENGRICHE Tidiane (ER-2037)
SZCZUBLIŃSKA Joanna Wiktoria (156070)



Contents

1	Introduction	1
2	Pseudocode	2
3	Plots	6
4	Results	10
5	Conclusion	11
6	All Other Results	12

1 Introduction

The purpose of this report is to explore advanced metaheuristics that overcome the limitations of multi-run local search algorithms. We implement a metaheuristic approach: **Large Neighborhood Search (LNS)**.

LNS iteratively explores large neighborhoods by partially destroying a current solution and repairing it, allowing the search to escape local optima while focusing on promising regions of the solution space.

The basic LNS procedure involves:

- Generating an initial solution and applying local search to improve it.
- Iterating until the average time of MSLS from the previous assignment is reached.

At each iteration:

- A **destroy operator** removes a fraction of nodes or edges from the solution, introducing diversification.
- A **repair operator** then reconstructs a feasible solution using greedy heuristics.
- Optionally, local search can be applied to improve the repaired solution.

The best solution is updated if the newly constructed solution improves the objective function, and this process repeats until the stopping condition is met.

In this work, we implement two variants of LNS: one that applies local search after each destroy-repair step, and one that does not. The destroy operator is designed to remove approximately **30%** of nodes or edges, selecting costly elements with higher probability while incorporating randomization to maintain diversification. The repair operator uses the best greedy heuristic from previous assignments, including **weighted k-regret strategies** (greedy-regret with $k=2$ and weight 0.5), to efficiently reconstruct solutions. Steepest local search is applied to the initial solution in both variants, and optionally to the repaired solutions.

The performance of the LNS variants is compared against standard methods, including multi-start local search (MSLS), iterated local search (ILS), basic local search and the best greedy heuristics. Computational experiments are conducted over multiple runs, recording objective values, running times, and the number of main loop iterations for ILS and NLS. This comparative analysis highlights the effectiveness of large neighborhood search and the impact of incorporating local search within the destroy-repair workflow.

You can see the repository of our assignment on github:

https://github.com/tbengric/EC_laboratory

2 Pseudocode

In this section, we will present the pseudocodes of the algorithm. For this assignment, we will code in C++.

Struct Item

STRUCT Item:

idx: INTEGER // index of node in solution

weight: DOUBLE // weighted cost of node + edge

Destroy Solution

```
FUNCTION DESTROY_SOLUTION(solution, dist_matrix, nodes, rng,
fraction = 0.3):
  current ← solution
  n ← LENGTH(current)
  // Build weighted items (nodes + edges)
  items ← EMPTY_LIST of Item
  FOR i FROM 0 TO n-1 DO :
    u ← current[i]
    v ← current[(i+1) mod n]
    edge_cost ← dist_matrix[u][v]
    node_cost ← nodes[u].cost
    ADD Itemidx=i, weight=edge_cost + node_cost TO items
  target ← MAX(1, ROUND(n * fraction))
  removed ← EMPTY_SET
  seg_len_dist ← UNIFORM_INT(2,6)
  mode_dist ← UNIFORM_INT(1,2)
  offset_dist ← UNIFORM_INT(0, n/2)
  WHILE SIZE(removed) < target DO :
    total_weight ← SUM(weights of items)
    r ← UNIFORM_REAL(0, total_weight)
    cum ← 0, start_idx ← 0
    FOR it IN items DO :
      cum ← cum + it.weight
      IF cum ≥ r AND it.idx NOT IN removed THEN :
        start_idx ← it.idx
        BREAK
    mode ← RANDOM(mode_dist)
    IF mode = 1 THEN : // single subpath
      len ← RANDOM(seg_len_dist)
      FOR i FROM 0 TO len-1 AND SIZE(removed) < target DO :
        ADD (start_idx + i) mod n TO removed
    ELSE IF mode = 2 THEN : // multiple subpaths
      num_sp ← RANDOM(2,3)
      FOR s FROM 1 TO num_sp AND SIZE(removed) < target DO :
        len ← RANDOM(seg_len_dist)
        off ← RANDOM(offset_dist)
        FOR j FROM 0 TO len-1 AND SIZE(removed) < target DO :
          ADD (start_idx + off + j) mod n TO removed
  // Build resulting solution
  new_solution ← EMPTY_LIST
  FOR i FROM 0 TO n-1 DO :
    IF i NOT IN removed THEN ADD current[i] TO new_solution
  RETURN new_solution
```

Repair Solution

```
FUNCTION REPAIR_SOLUTION(partial_solution, dist, nodes,
kRegret=2, weightRegret=0.5):
  weightObjective  $\leftarrow$  1.0 - weightRegret
  total_nodes  $\leftarrow$  LENGTH(nodes)
  numToSelect  $\leftarrow$  CEIL(total_nodes / 2)
  path  $\leftarrow$  partial_solution
  visited  $\leftarrow$  [False]*total_nodes
  SET nodes FROM path in visited as TRUE
  WHILE LENGTH(path) < numToSelect + 1 DO :
    bestNode  $\leftarrow$  -1, bestPos  $\leftarrow$  -1, bestWeightedScore  $\leftarrow$  -INF
    FOR node IN nodes DO :
      IF node.id ALREADY visited THEN CONTINUE
      insertionCosts  $\leftarrow$  EMPTY_LIST
      FOR i FROM 0 TO LENGTH(path)-2 DO :
        u  $\leftarrow$  path[i], v  $\leftarrow$  path[i+1]
        cost  $\leftarrow$  dist[u][node.id] + dist[node.id][v] - dist[u][v] + node.cost
        ADD (cost, i+1) TO insertionCosts
      SORT insertionCosts BY cost ASCENDING
      regret  $\leftarrow$  SUM of first kRegret differences in insertionCosts
      weightedScore  $\leftarrow$  weightRegret*regret -
weightObjective*insertionCosts[0].cost
      IF weightedScore > bestWeightedScore THEN :
        bestWeightedScore  $\leftarrow$  weightedScore
        bestNode  $\leftarrow$  node.id
        bestPos  $\leftarrow$  insertionCosts[0].position
      INSERT bestNode AT bestPos IN path
      SET bestNode in visited as TRUE
  RETURN path
```

Large Neighborhood Search (LNS)

```
FUNCTION LNS(initial, dist, nodes, remove_fraction, avg_msls_time, timer,
useLocalSearch):
  current  $\leftarrow$  STEEPEST_LOCAL_SEARCH(initial, dist, "edge", nodes)
  best  $\leftarrow$  current, best_score  $\leftarrow$  COMPUTE_OBJECTIVE(current, dist, nodes)
  current_score  $\leftarrow$  best_score
  iterations  $\leftarrow$  0
  WHILE timer < avg_msls_time DO :
    iterations  $\leftarrow$  iterations + 1
    partial  $\leftarrow$  DESTROY_SOLUTION(current, dist, nodes, rng,
remove_fraction)
    repaired  $\leftarrow$  REPAIR_SOLUTION(partial, dist, nodes)
    IF useLocalSearch THEN :
      repaired  $\leftarrow$  STEEPEST_LOCAL_SEARCH(repaired, dist, "edge", nodes)
      repaired_score  $\leftarrow$  COMPUTE_OBJECTIVE(repaired, dist, nodes)
      IF repaired_score < current_score THEN :
        current  $\leftarrow$  repaired
        current_score  $\leftarrow$  repaired_score
        IF current_score < best_score THEN :
          best  $\leftarrow$  current
          best_score  $\leftarrow$  current_score
  RETURN best
```

3 Plots

After trying this algorithm, we will check its effectiveness by showing the constructed cycle.

In the plot, the cost of each node is represented by its size: larger nodes indicate a higher cost, while smaller ones indicate a lower cost. Additionally, all possible nodes are displayed, but the nodes chosen for the path are shown in blue, with their IDs printed on them. The started node is pointed with red color.

Figures for Steepest Local Search for Random Solution

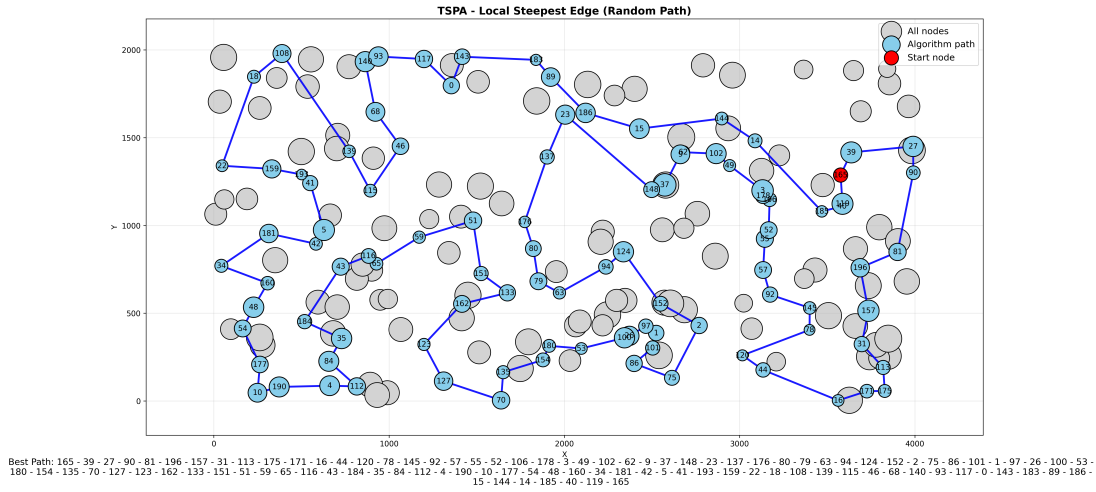


Figure 1: Local Steepest Edge (Random Path) - TSPA

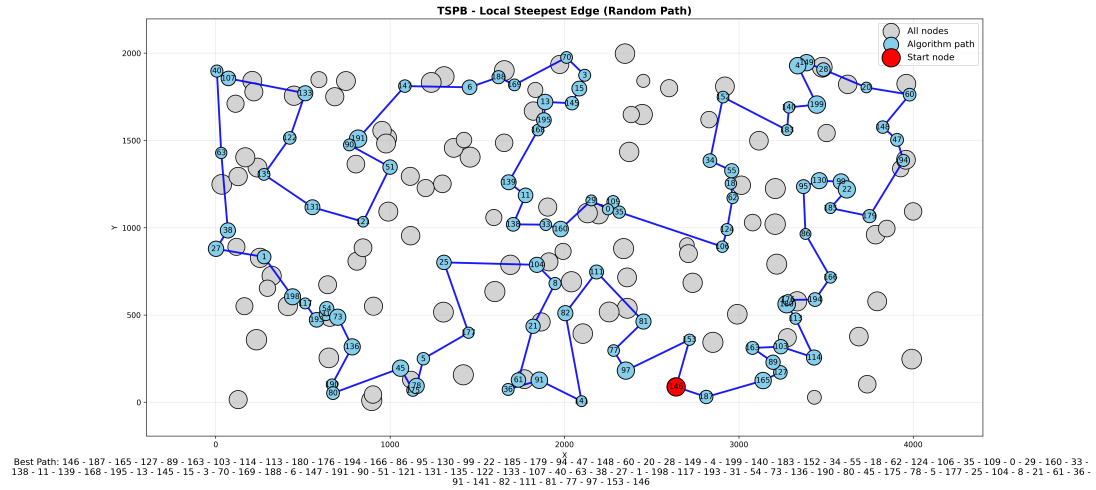


Figure 2: Local Steepest Edge (Random Path) - TSPB

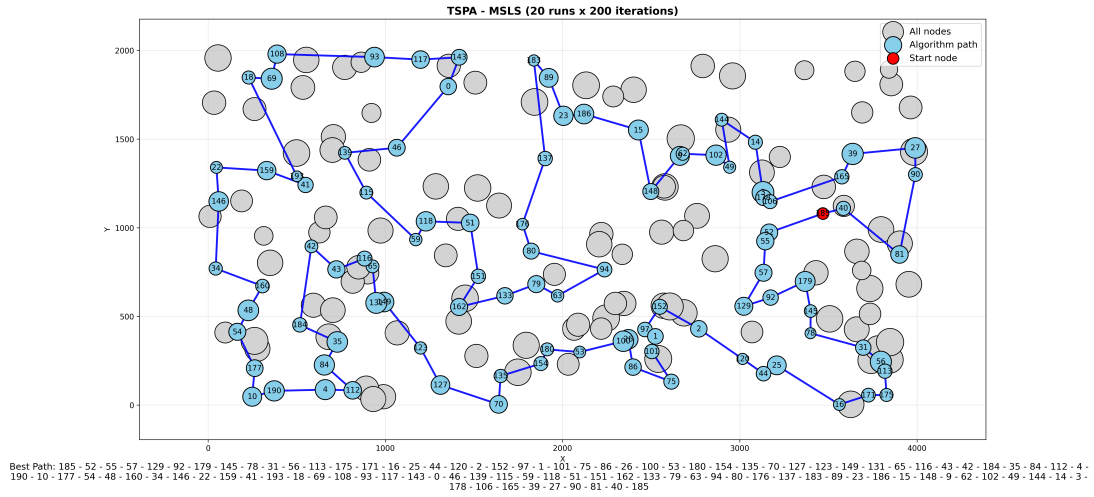


Figure 3: Multiple Start Local Search (MSLS) - TSPA

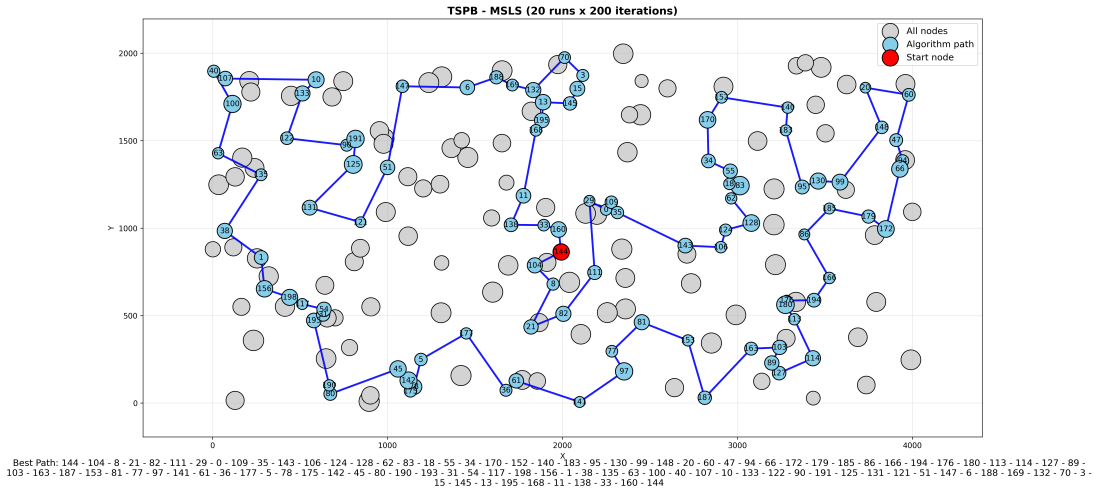


Figure 4: Multiple Start Local Search (MSLS) - TSPB

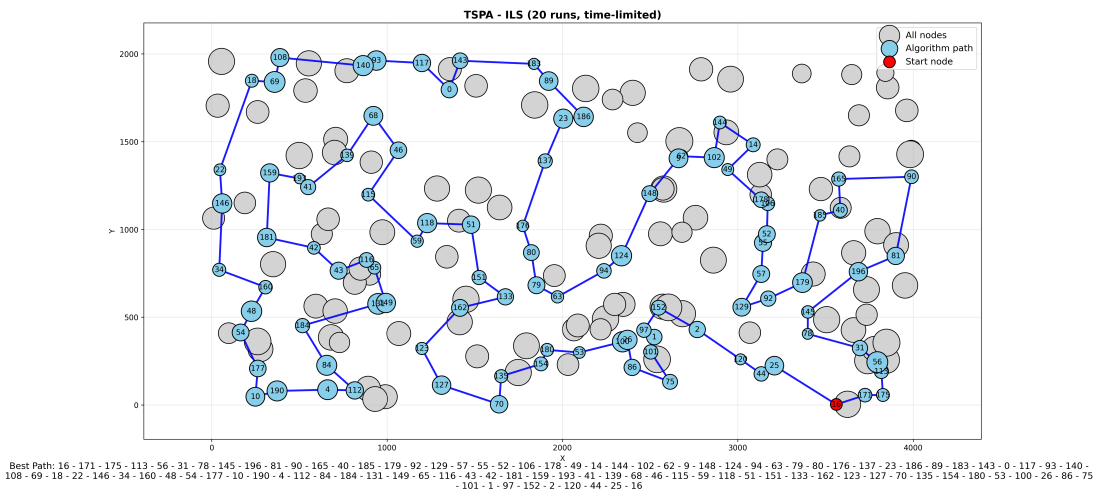


Figure 5: Iterated Local Search (ILS) - TSPA

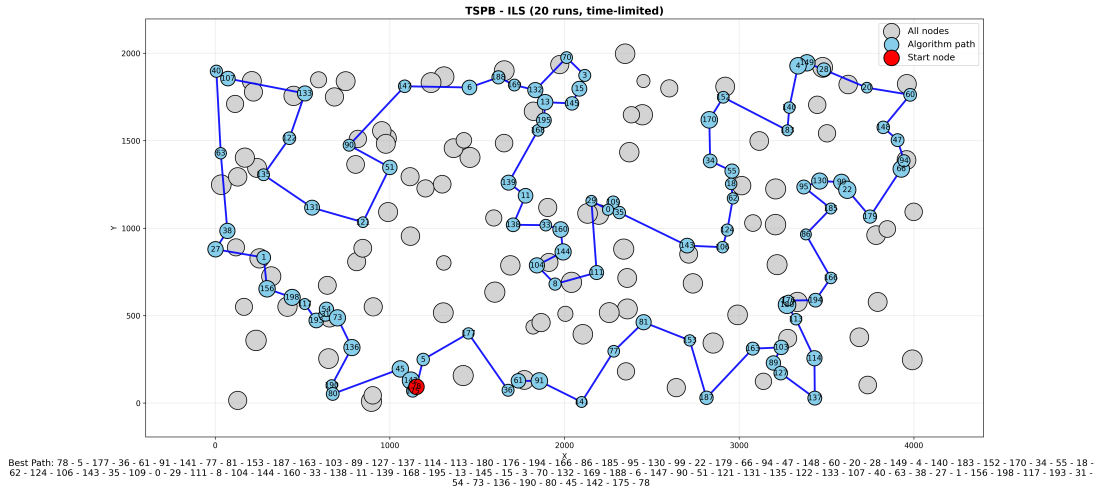


Figure 6: Iterated Local Search (ILS) - TSPB

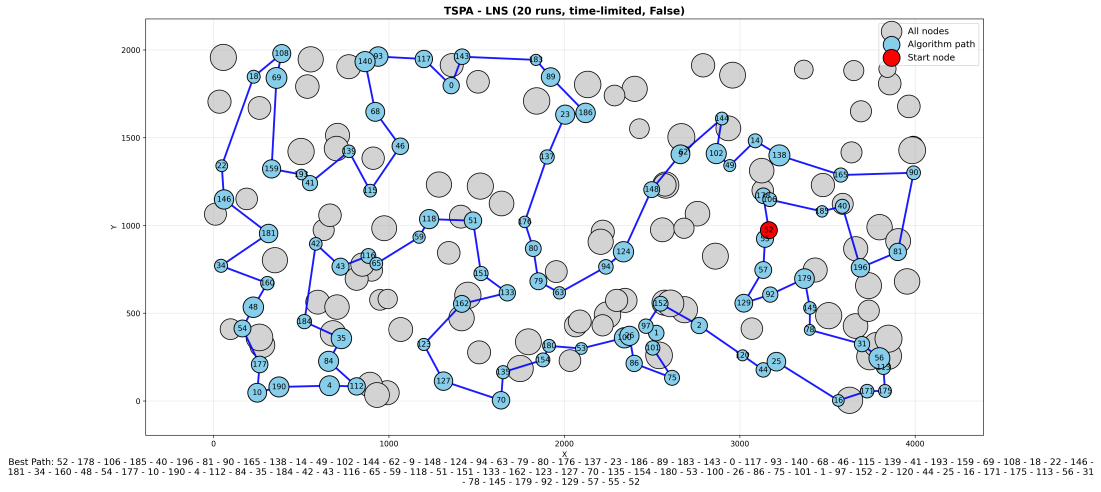


Figure 7: Large Neighborhood Search (LNS), useLocalSearch = False - TSPA

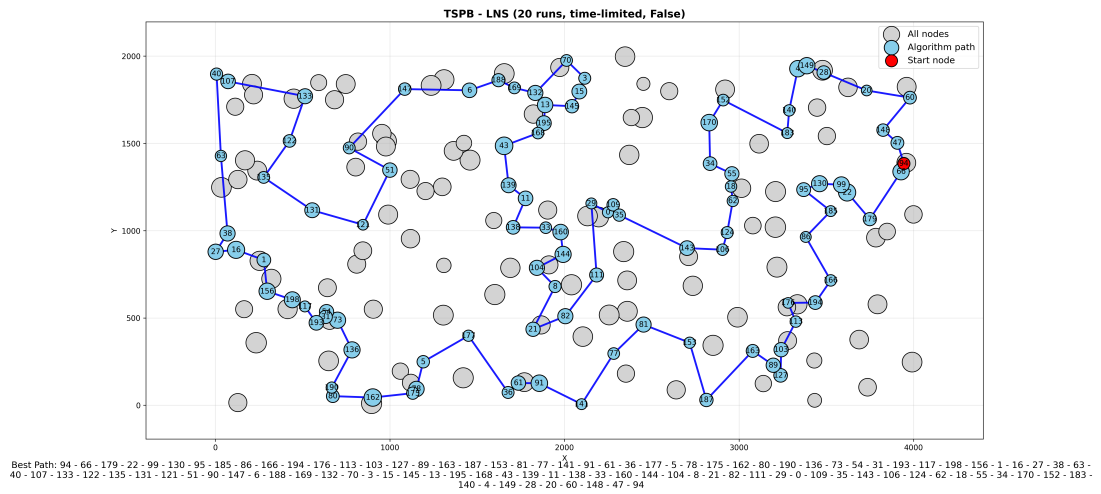


Figure 8: Large Neighborhood Search (LNS), useLocalSearch = False - TSPB

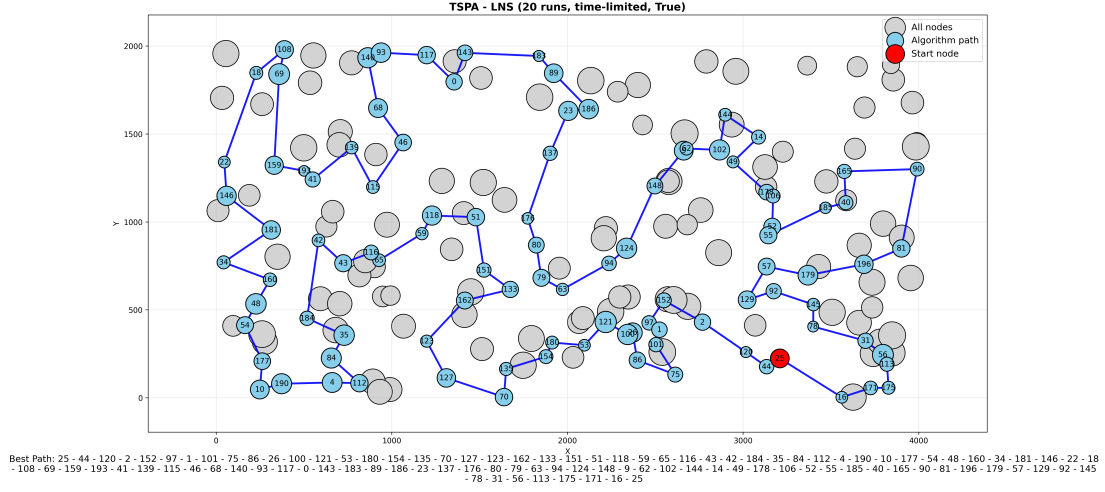


Figure 9: Large Neighborhood Search (LNS), useLocalSearch = True - TSPA

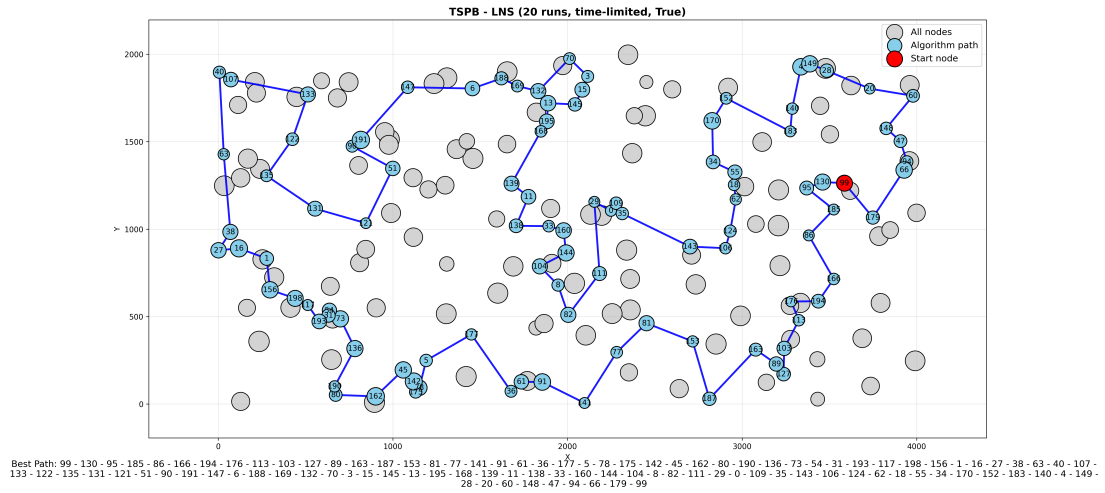


Figure 10: Large Neighborhood Search (LNS), useLocalSearch = True - TSPB

4 Results

Method	TSPA (Avg, Min, Max)	TSPB (Avg, Min, Max)
Random Path	265530.42 (229719, 294157)	213673.99 (195590, 234518)
Local Steepest Edge (Random Path)	74200.79 (71517, 77754)	48421.32 (45631, 52371)
Local Steepest Edge with LM (Random Path)	74123.34 (71525, 78562)	49044.82 (46069, 52197)
MSLS (20 runs x 200 iterations)	71365.90 (70485, 71843)	45905.95 (45389, 46366)
ILS (20 runs, time-limited)	69627.05 (69141, 70453)	43908.50 (43497, 44459)
NLS (20 runs, time-limited, True)	69543.00 (69207, 69904)	43996.75 (43581, 44260)
NLS (20 runs, time-limited, False)	69746.00 (69263, 70633)	44256.45 (43671, 45105)
Local Steepest Edge (Candidates, k=15, Random Path)	75607.94 (72031, 80451)	48478.01 (45923, 51750)
Local Steepest Edge (Random Path)	72687.00 (71034, 77924)	49190.00 (45537, 51303)

Table 1: Comparison of TSPA and TSPB results

Method	TSPA Time (avg, min, max) [μs]	TSPB Time (avg, min, max) [μs]
Random Path	0.0036 (0.0033, 0.0176)	0.0052 (0.0044, 0.0239)
Local Steepest Edge (Random Path)	9.7100 (8.1613, 13.9574)	10.2649 (8.0421, 15.3924)
Local Steepest Edge with LM (Random Path)	9.3588 (6.6432, 12.3063)	9.3000 (7.0456, 12.0311)
MSLS (20 runs x 200 iterations)	1706.4574 (1689.2280, 1791.0236)	1900.8621 (1791.4707, 2033.1554)
ILS (20 runs, time-limited)	1706.6590 (1706.4574, 1707.1652)	1901.0305 (1900.8718, 1901.2089)
NLS (20 runs, time-limited, True)	1709.7860 (1707.4118, 1711.7095)	1903.8503 (1901.1062, 1907.0236)
NLS (20 runs, time-limited, False)	1709.8836 (1706.8554, 1712.9383)	1903.7007 (1901.0532, 1906.1929)
Local Steepest Edge (Candidates, k=15, Random Path)	2.1406 (1.8998, 3.0473)	2.3134 (1.9769, 3.4469)
Local Greedy Edge (Random Path)	58.0691 (47.6349, 99.7285)	52.4021 (45.1037, 84.4763)

Table 2: Average, minimum, and maximum execution times for TSPA and TSPB

Metric	LNS (True)	LNS (False)
Average Main Loop per Run	271.20	279.75
Min Main Loop	266	257
Max Main Loop	291	290

Table 3: Number of iterations of the main loop in TSPA

Metric	LNS (True)	LNS (False)
Average Main Loop per Run	311.45	319.65
Min Main Loop	304	294
Max Main Loop	322	329

Table 4: Number of iterations of the main loop in TSPB

5 Conclusion

Based on the results presented in Tables 1, 2, 3, and 4, the following observations can be made:

ILS and NLS(True) are the best performing algorithms in terms of objective value. Both have limited running times equal to the average time of MSLS, so their time execution is almost the same.

Depending on the TSP instance, ILS performs slightly better in TSPB, while NLS(True) performs better in TSPA, and overall their results are very similar.

We can also observe that NLS(True) outperforms NLS(False). This indicates that applying an additional steepest local search after the repair operations improves the final solution quality. Since the running time remains the same as before, using this steepest local search with edge (True) is better option.

Regarding the number of iterations of the main loop, we can observe that the algorithm with additional local search LNS(True), performs fewer main-loop iterations, as part of the available time is spent on executing the extra local search step.

6 All Other Results

Method	TSPA Avg (Min, Max)	TSPB Avg (Min, Max)
Random Path	264188.61 (238257, 293017)	213973.45 (184171, 238914)
Nearest Neighbor (End)	85108.51 (83182, 89433)	54390.43 (52319, 59030)
Nearest Neighbor (Flexible)	73594.10 (71868, 75953)	48694.25 (44609, 57283)
Greedy Cycle	72635.98 (71488, 74410)	51400.60 (49001, 57324)
Greedy 2-Regret	115400.24 (105692, 126860)	72712.85 (67809, 78406)
Greedy 2-Regret (w=0.5)	72136.15 (71108, 73395)	50934.60 (47144, 55700)
NN-Flex 2-Regret	117138.49 (108151, 124921)	74444.46 (69933, 80278)
NN-Flex 2-Regret (w=0.5)	72407.59 (70010, 75452)	47664.46 (44891, 55247)
Local Greedy Node (Greedy Cycle)	70687.00 (70687, 70687)	46443.67 (46328, 51512)
Local Greedy Edge (Greedy Cycle)	70618.84 (70571, 70663)	46089.89 (45824, 51482)
Local Steepest Node (Greedy Cycle)	70687.00 (70687, 70687)	46466.54 (46371, 51512)
Local Steepest Edge (Greedy Cycle)	70571.00 (70571, 70571)	45996.84 (45867, 51259)
Local Greedy Node (Random Path)	85217.85 (77570, 94277)	60178.87 (54006, 66885)
Local Greedy Edge (Random Path)	74232.03 (71777, 77750)	48192.19 (46109, 51934)
Local Steepest Node (Random Path)	85271.00 (85271, 85271)	63491.00 (63491, 63491)
Local Steepest Edge (Random Path)	72687.00 (72687, 72687)	49190.00 (49190, 49190)
Local Steepest (Candidates, k=5)	88563.00 (88563, 88563)	52093.00 (52093, 52093)
Local Steepest (Candidates, k=10)	83598.00 (83598, 83598)	51153.00 (51153, 51153)
Local Steepest (Candidates, k=15)	81136.00 (81136, 81136)	48854.00 (48854, 48854)
Local Steepest Edge with LM (Random Path)	74187.41 (71646, 78764)	49144.47 (45888, 53039)

Table 5: Comparison of average, minimum, and maximum objective values for TSPA and TSPB. The 3 lowest average values in each column are highlighted in bold.

Method	TSPA Time Avg (Min, Max)	TSPB Time Avg (Min, Max)
Random Path	0.0036 (0.0034, 0.0144)	0.0039 (0.0036, 0.0232)
Nearest Neighbor (End)	0.0694 (0.0463, 0.1093)	0.0685 (0.0470, 0.1282)
Nearest Neighbor (Flexible)	91.0993 (85.8761, 119.1184)	94.7285 (87.4900, 154.2259)
Greedy Cycle	87.6467 (82.1853, 135.8448)	90.6419 (84.6333, 153.7690)
Greedy 2-Regret	13.4090 (12.2387, 18.2542)	13.4612 (12.4574, 21.9434)
Greedy 2-Regret (w=0.5)	12.9063 (11.8320, 18.2641)	12.6926 (11.2819, 20.7204)
NN-Flex 2-Regret	13.8750 (12.7669, 18.4833)	13.9626 (12.9940, 21.1062)
NN-Flex 2-Regret (w=0.5)	13.5540 (12.4458, 19.2731)	13.6345 (12.6257, 21.8523)
Local Greedy Node (Greedy Cycle*)	1.2763 (0.8682, 2.4062)	1.8935 (1.1700, 5.2516)
Local Greedy Edge (Greedy Cycle*)	1.5676 (1.1047, 3.2752)	2.5081 (1.3700, 5.8678)
Local Steepest Node (Greedy Cycle*)	0.5265 (0.4719, 1.0174)	0.7846 (0.4909, 1.6765)
Local Steepest Edge (Greedy Cycle*)	0.5418 (0.4837, 1.0700)	1.0705 (0.6829, 2.2377)
Local Greedy Node (Random Path)	57.7227 (44.0444, 123.7979)	51.7444 (41.5894, 92.8947)
Local Greedy Edge (Random Path)	58.0691 (47.6349, 99.7285)	52.4021 (45.1037, 84.4763)
Local Steepest Node (Random Path)	10.9774 (10.0351, 18.7647)	13.6565 (12.4567, 26.2057)
Local Steepest Edge (Random Path)	9.1549 (8.5878, 12.7278)	9.0840 (8.4419, 18.6255)
Local Steepest (Candidates, k=5)	1.4490 (1.3910, 1.9627)	1.4250 (1.3467, 1.8863)
Local Steepest (Candidates, k=10)	1.8156 (1.7346, 2.3018)	1.7992 (1.7277, 2.4961)
Local Steepest (Candidates, k=15)	2.1838 (2.0927, 2.7651)	2.1374 (2.0609, 2.9937)
Local Steepest Edge with LM (Random Path)	50.6919 (35.5602, 69.0622)	53.7797 (39.3278, 92.4221)

Table 6: Comparison of average, minimum, and maximum execution times in microseconds for TSPA and TSPB. Greedy Cycle* is the Greedy Cycle 2-Regret (w=0.5)